매달 반복되는 일을 한 방에: 자동화 파이프라인

6주차 실습

소요시간: 약 115분 | week04 엑셀 + week05 DB 경험 필요

Presented by 김형진

지금까지 한 것

week04

법인 엑셀 8개 → Python으로 합치기 → 결과 엑셀

↓

week05

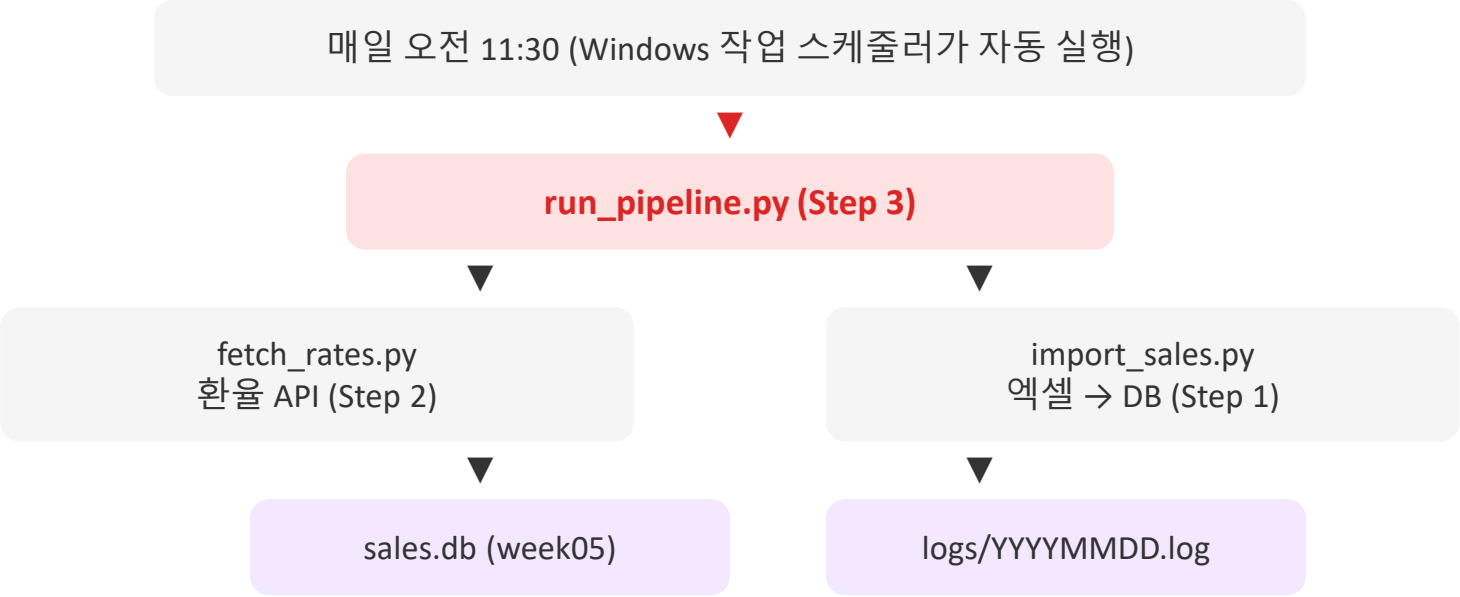
같은 데이터를 DB에 → AI가 SQL 조회 → 웹 대시보드

↓

week06

이 모든 과정을 **한 번의 명령으로, 매일 자동으로**

오늘 만들 것



week05 대시보드 새로고침 → 최신 데이터가 바로 보임

오늘 배울 3가지 개념

1. 멱등성 (Idempotency)

"같은 걸 몇 번 돌려도 결과가 같아야 한다"
실수로 두 번 돌려도 DB가 오염되지 않는 안전장치

2. 외부 API

"우리 DB 바깥의 데이터를 요청으로 받아오기"
한국수출입은행에 "오늘 환율 주세요" 라고 요청을 보냄

3. 스케줄 / 주기 실행

"내가 PC 앞에 없어도 컴퓨터가 알아서 돌림"
Windows 작업 스케줄러에 등록만 하면 매일 자동 실행

Step 1
Excel → DB 자동 반영

Claude에게 시키면 멱등성까지 알아서 챙겨줍니다

import_sales.py

data/incoming/ 폴더의 법인 엑셀 8개를 DB에 자동 반영

① 폴더 스캔
법인_*.xlsx 8개 읽기

→

② 법인코드 파싱
법인_US01_미국.xlsx
→ US01

→

③ DB UPSERT
(corp_code, month)
→ 덮어쓰기

Claude에게 이렇게 말하면 알아서 만들어줍니다:

```
data/incoming/ 폴더의 법인 엑셀 8개를 읽어서
monthly_sales에 반영하는 import_sales.py를 만들어줘.

중요: 같은 (법인코드, 월) 조합이 이미 있으면 덮어쓰기.
두 번 돌려도 행이 중복되지 않아야 해.
```

멱등성 (Idempotency)

자동화의 핵심 안전장치

"같은 걸 몇 번 돌려도 결과가 같아야 한다"

재경팀에서 제일 무서운 실수:
"저 혹시 이거 한 번 더 돌려도 돼요?"

멱등성이 지켜지면 → **돌려도 된다**
안 지켜지면 → 안 된다 (매출이 두 배로 뻥튀기)

UPSERT = INSERT ... ON CONFLICT DO UPDATE
"있으면 덮어쓰고, 없으면 넣는다" — Claude가 CLAUDE.md 보고 **알아서 적용**

멱등성 시연

같은 스크립트를 두 번 돌려도 결과가 같다

1회차 실행

```
처리 파일: 8개
처리 행:   48건 (UPSERT)
monthly_sales 총 행 수: 48
```

2회차 실행

```
처리 파일: 8개
처리 행:   48건 (UPSERT)  ← 그대로!
monthly_sales 총 행 수: 48
```

핵심: 총 행 수가 그대로. 같은 엑셀을 실수로 10번 돌려도 DB는 멀쩡.
"실수로 한 번 더 돌렸는데 괜찮은지" 걱정할 필요 없음 = 멱등성!

Step 1 시연 가이드

Claude에게 순서대로 시킵니다

① 스크립트 생성 + 실행

```
import_sales.py를 실행해서 결과를 보여줘.
```

② 멱등성 검증 (핵심!)

```
import_sales.py를 한 번 더 돌려줘.
monthly_sales의 행 수가 그대로인지 확인해줘.
```

③ 데이터 변경 시뮬레이션 (시간 남으면)

```
미국법인의 2026-03 금액을 200000으로 바꾸고
import_sales.py를 다시 돌려줘.
DB에 잘 반영됐는지 SELECT로 확인해줘.
```

UNIQUE 인덱스

DB가 지켜주는 안전장치

```
CREATE UNIQUE INDEX ux_sales_corp_month
ON monthly_sales(corp_code, month);
```

쉽게 말하면:

"**corp_code**랑 **month**가 같은 행은 DB에 **오직 한 개**만 존재할 수 있다."

이 규칙이 있으니까 **"똑같은 건 자동으로 거부되거나 업데이트된다"**가 보장됩니다.
이것도 Claude가 CLAUDE.md 보고 **알아서 넣어줍니다.**

Step 2
환율 API 자동 갱신

Claude가 API 함정까지 알아서 처리합니다 — CLAUDE.md에 규칙만 적으면 됩니다

한국수출입은행 오픈API

외부 서버에 데이터를 요청해서 받아오기

항목	내용
엔드포인트	exchangeJSON
인증	authkey (무료 발급)
파라미터	searchdate (YYYYMMDD), data=AP01
응답	JSON 배열 — 통화별 매매기준율
사용 라이브러리	**urllib.request (표준 라이브러리만!)**

Claude에게 이렇게 말하면 알아서 만들어줍니다:

```
한국수출입은행 오픈API에서 오늘 환율을 받아와서
exchange_rates에 저장하는 fetch_rates.py 만들어줘.
규칙은 CLAUDE.md에 있어.
```

현실 함정 ① ②

API가 주는 데이터를 "그대로 쓰면 안 되는" 이유

함정 ① JPY는 100엔 단위로 온다

API 응답	의미	우리가 해야 할 것
JPY(100) = 920.00	100엔에 920원	**÷ 100 → JPY = 9.20**

안 하면? 환율이 100배 튀는 대형 사고!

함정 ② CNY가 CNH로 온다

API가 주는 값	우리 DB에 저장할 값	이유
CNH (역외위안)	**CNY (위안)**	사실상 동일 환율

교훈: 같은 통화도 시장·거래소에 따라 다른 코드. 매핑 규칙을 하네스에 적어야!

현실 함정 ③ ④

"없으면 뭘 할지"를 미리 정해둬야 한다

함정 ③ VND(베트남 동)는 API에 없다

폴백 규칙:

1. DB에 있는 가장 최근 VND 환율을 재사용
2. 그것도 없으면 하드코딩 상수 **0.056** 사용
3. 로그에 "VND: 폴백값 사용" 경고

함정 ④ 주말·공휴일·오전 11시 이전 = 빈 응답

토요일에 호출하면: **[ ]** (빈 배열) — 에러 아님, "데이터 없음"

폴백: 직전 영업일 환율을 오늘 날짜로 복사 + 로그 기록

교훈: *이 4가지 함정은 Claude가 CLAUDE.md 보고 알아서 처리합니다!*

Step 2 시연 가이드

Claude에게 순서대로 시킵니다

① 환경 설정

```
.env.example을 .env로 복사해줘.
키는 (강사가 공유한 키) 로 설정해줘.
```

② 스크립트 생성 + 실행

```
fetch_rates.py를 실행해서 결과 보여줘.
```

③ 주말 폴백 시뮬레이션 (시간 남으면)

```
fetch_rates.py를 searchdate=20260104 (일요일)로
돌려봐. 폴백 로직이 잘 동작하는지 확인해줘.
```

Step 3
파이프라인 묶기 + 로그

Claude에게 "하나로 묶고 로그 남겨줘"라고 말하면 끝

run_pipeline.py

Claude가 만들어주는 파이프라인 오케스트레이터

지금 상태:

```
py fetch_rates.py
py import_sales.py
```

묶은 후:

```
py run_pipeline.py
```

Claude가 만들어주는 구조:

- fetch_rates.py, import_sales.py를 **함수로 import**해서 호출
  (subprocess 아님 — 에러 메시지 전달이 편함)
- 환율 갱신 → 매출 import 순서대로 실행
- 한 단계 실패해도 다음 단계는 진행 (독립적)
- 여러분은 Claude에게 **"묶어줘"**라고 말하면 됩니다

로그: 자동화의 생명줄

내가 안 볼 때 돌아가는 시스템은 로그가 전부

로그 없으면: "뭐가 문제야?" → "모르겠어요" (일주일 뒤 발견)

로그 있으면: "이 날짜 로그 보여줘" → 원인 확인 3초

logs/20260409.log:

```
===== 2026-04-09 09:00:12 파이프라인 시작 =====
[환율 갱신] 시작
[환율 갱신] USD=1,350  JPY=9.20  CNY=186  VND=0.056 (폴백)
[환율 갱신] 완료 (1.2초)
[매출 import] 시작
[매출 import] 8개 파일: UPSERT 48건 (총 48행)
[매출 import] 완료 (0.3초)
===== 2026-04-09 09:00:13 파이프라인 완료 (1.5초) =====
```

실패 복구: "그냥 다시 돌리면 된다"

멱등성 덕분에 복구가 간단합니다

시나리오: 어제 환율 API가 장애 → 오늘 복구

1. 어제: [환율 갱신] 실패 로그 / [매출 import] 정상 완료
   → exit code 1 / failure flag 생성

2. 오늘: .env 키 복구 후 **run_pipeline.py 다시 실행**
   → 환율 정상 갱신
   → 매출 행 수 **동일** (멱등성!)
   → exit code 0 / failure flag 자동 삭제

멱등성은 "편의 기능"이 아니라 복구 가능성의 근간

"실수로 한 번 더 돌려도 괜찮다" → 자동화가 무서운 것에서 편한 것으로

Step 3 시연 가이드

Claude에게 순서대로 시킵니다

① 파이프라인 생성 + 실행

```
run_pipeline.py를 돌리고,
생성된 로그 파일을 보여줘.
```

② 로그 누적 확인

```
run_pipeline.py를 3번 연속으로 돌려줘.
logs/ 파일이 덮어쓰여졌는지, append 됐는지 확인해줘.
```

③ 일부러 실패시켜보기

```
.env의 KOREAEXIM_AUTHKEY를 일부러 잘못된 값으로 바꾸고
run_pipeline.py를 돌려줘. 로그와 exit code를 보여줘.
끝나면 .env를 원래대로 복구해줘.
```

Step 4 & 5
스케줄 + 모니터링

등록 스크립트도 Claude가 만들어줍니다. 여러분은 실행만.

Windows 작업 스케줄러

매일 오전 11:30에 자동 실행

왜 오전 11:30인가?

- 한국수출입은행 환율은 **평일 오전 11:00 이후** 게시
- 11:00에서 30분 여유 → 게시 지연·네트워크 지연 커버
- 스케줄 시각 = 데이터가 언제 준비되는지부터 확인!

등록 스크립트 핵심:

```
$TaskName = "Week06Pipeline"
$Action   = New-ScheduledTaskAction
              -Execute $PyPath
              -Argument $Script
              -WorkingDirectory $WorkDir
$Trigger  = New-ScheduledTaskTrigger -Daily -At 11:30am
Register-ScheduledTask -TaskName $TaskName ...
```

모니터링: 실패를 눈에 보이게

자동화의 무서운 점 — 내가 안 볼 때 돌아간다

로그는 "이미 문제를 알고 있을 때" 원인을 찾는 도구.

문제를 발견하는 도구는 따로 필요합니다.

파이프라인 실패 시:

1. data/failure_YYYYMMDD.flag 파일 생성
2. 바탕화면에 '파이프라인_실패_YYYYMMDD.txt' 요약 파일
   → 다음 날 아침 PC 켤 때 눈에 들어옴
3. week05 대시보드 상단에 **빨간 배너** 표시

파이프라인 성공 시 (복구):

- failure flag 자동 삭제
- 대시보드 빨간 배너 사라짐

핵심: 이미 보고 있는 곳(대시보드)에 알리기!

Step 4 & 5 시연 가이드

Claude에게 순서대로 시킵니다

① 스케줄러 등록 스크립트 생성

```
run_pipeline.py를 매일 오전 11:30에 자동 실행하는
Windows 작업 스케줄러 등록용 PowerShell 스크립트를 만들어줘.
Task 이름은 Week06Pipeline.
```

② 모니터링 기능 추가

```
run_pipeline.py에 실패 시 failure flag 파일 생성하고
바탕화면에 알림 txt 만드는 기능 추가해줘.
성공하면 flag 자동 삭제.
```

③ 실패→복구 시연

```
.env 키를 일부러 틀리게 → 실행 → flag 확인 →
키 복구 → 재실행 → flag 삭제 확인
```

오늘의 핵심 정리

배운 것	한 줄 요약
멱등성	같은 걸 여러 번 돌려도 안전해야 한다
외부 API	남이 주는 데이터는 절대 "그대로" 쓰지 마라
로그	내가 안 볼 때 돌아가는 시스템은 로그가 전부다
스케줄	사람이 누르지 않아도 돌아가게 만드는 게 자동화의 완성

다음 시간 과제: 내 업무 중 하나를 자동화해 오세요.
"매달/매주/매일 반복되는 것" → "입력은 어디서?" → "출력은 어디로?"

E.O.D